Лабораторная работа № 6

Тема: CRUD-операции с использованием Prisma ORM 7 в существующем Express REST API

Цель: Заменить ручные SQL-запросы через pg на Prisma Client в Product Service, реализовать полный CRUD, сохранить валидацию, HTTP-статусы и связи Product–Category.

Дисциплина: РО 4.3. Интеграция серверных приложений с реляционными базами данных

Лабораторная работа № 6. CRUD-операции с использованием Prisma ORM 7

Тема: Замена библиотеки pg на Prisma Client внутри существующего REST API магазина. Полный CRUD товаров с сохранением архитектуры Route → Controller → Service → PostgreSQL, валидации запросов и корректных HTTP-ответов.

Цель: реализовать методы create(), findMany(), findUnique(), update(), delete() в Product Service, связать товары с категориями, проверить POST, GET, PATCH, DELETE и ошибки клиента.

> **Исходные условия**
>
> Это продолжение ЛР № 5. Prisma 7 уже настроена, модели Product и Category получены из PostgreSQL, Prisma Client сгенерирован. Не создавайте проект и базу заново, не удаляйте существующие таблицы.

1. Проверьте исходный проект

- Node.js 22; Express 5; package.json содержит «type»: «module».
- Таблицы PostgreSQL: products(id, name, price, in_stock, category_id) и categories(id, name).
- products.category_id — NOT NULL и FOREIGN KEY на categories.id; текстового поля products.category нет.
- Файлы config/prisma.js, prisma/schema.prisma, prisma.config.ts (или prisma7.config.ts), generated/prisma доступны после ЛР № 5.
- В существующем проекте работают маршруты /api/products и контроллеры.

Ожидаемая архитектура: HTTP → productRoutes.js → productController.js → productService.js → config/prisma.js → PostgreSQL. Только Service должен знать о Prisma.

2. Проверьте модель Prisma и сгенерируйте клиент

Файл: prisma/schema.prisma. Проверьте модели и отображение имён реальных таблиц через @@map, а имён столбцов — через @map. Не удаляйте другие модели, если они уже есть.

Файл prisma/schema.prisma — модели из ЛР № 5

```
model Category {
  id       Int       @id
  name     String    @db.VarChar(100)
  products Product[]

  @@map("categories")
}

model Product {
  id         Int      @id @default(autoincrement())
  name       String   @db.VarChar(150)
  price      Int
```

```
  inStock    Boolean  @default(true) @map("in_stock")
  categoryId Int      @map("category_id")
  category   Category @relation(fields: [categoryId], references: [id], onDelete:
NoAction, onUpdate: NoAction, map: "fk_product_categorie")

  @@map("products")
}
```

Соответствие реальной БД

Если в вашей базе другое имя FOREIGN KEY, используйте значение из результата db pull. Если category.id имеет автоинкремент, сохраните реальное определение, полученное из БД. Эти модели — пример для указанной схемы.

```
npx prisma validate
npx prisma generate
```

Файл config/prisma.js из ЛР № 5 не изменяется: он экспортирует prisma. Для учебного JavaScript-проекта генератор Prisma 7 создаёт .ts-файлы, поэтому запуск через tsx остаётся обязательным в принятой конфигурации.

3. Сравните SQL и Prisma

REST	SQL через pg	Prisma Client
GET /api/products	SELECT ...	findMany()
GET /api/products/:id	SELECT ... WHERE id	findUnique()
POST /api/products	INSERT ... RETURNING	create()
PATCH /api/products/:id	UPDATE ... RETURNING	update()
DELETE /api/products/:id	DELETE ... RETURNING	delete()

В Prisma объект where указывает условие, data — новые значения, orderBy — сортировку, include — загрузку связанной модели. Результат возвращается сразу как объект JavaScript или массив, без result.rows.

4. Замените код Service на Prisma

Файл: services/productService.js. Замените обращения к pool.query() на Prisma Client. Обратите внимание: include получает связанную Category; название категории для прежнего API берётся из category.name.

Файл services/productService.js — все пять методов Prisma

```
import { prisma } from "../config/prisma.js";

function formatProduct(product) {
  if (!product) return null;
  const { category, ...fields } = product;
  return { ...fields, category: category.name };
}
```

```
// READ: список
export async function getAllProducts() {
  const products = await prisma.product.findMany({
    orderBy: { id: "asc" },
    include: { category: true }
  });
  return products.map(formatProduct);
}

// READ: одна запись по уникальному id
export async function findProductById(id) {
  const product = await prisma.product.findUnique({
    where: { id },
    include: { category: true }
  });
  return formatProduct(product);
}

// CREATE: данные нового товара
export async function addProduct(data) {
  const product = await prisma.product.create({
    data: {
      name: data.name,
      price: data.price,
      categoryId: data.categoryId,
      inStock: data.inStock ?? true
    },
    include: { category: true }
  });
  return formatProduct(product);
}

// UPDATE: изменяются только переданные поля
export async function updateProductById(id, changes) {
  const product = await prisma.product.update({
    where: { id },
    data: changes,
    include: { category: true }
  });
  return formatProduct(product);
}

// DELETE: возвращается удалённый объект
export async function deleteProductById(id) {
  const product = await prisma.product.delete({
    where: { id },
    include: { category: true }
  });
  return formatProduct(product);
}
```

Граница ответственности

data: changes безопасно использовать только после middleware, которое пропускает разрешённые поля и приводит типы. Ниже приведено такое middleware. Метод update() / delete() может вернуть ошибку P2025, если строки нет; это будет обработано централизованно.

5. Сохраните и уточните валидацию HTTP-запросов

Файл: middleware/productValidation.js. Если в предыдущем проекте уже есть эквивалентная валидация, сохраните её; проверьте, что она учитывает categoryId вместо старого строкового category. Ниже — полный учебный вариант, чтобы пример можно было воспроизвести.

Файл middleware/productValidation.js

```
const allowed = ["name", "price", "categoryId", "inStock"];

function badRequest(res, message) {
  return res.status(400).json({ message });
}

function validId(req, res, next) {
  const id = Number(req.params.id);
  if (!Number.isSafeInteger(id) || id <= 0) {
    return badRequest(res, "Invalid product ID");
  }
  req.productId = id;
  next();
}

function validateBody(req, res, next, partial) {
  const body = req.body;
  if (!body || typeof body !== "object" || Array.isArray(body)) {
    return badRequest(res, "Expected JSON object");
  }
  const keys = Object.keys(body);
  if (keys.length === 0) return badRequest(res, "Empty request body");
  if (keys.some(key => !allowed.includes(key))) {
    return badRequest(res, "Unknown product field");
  }
  if (!partial && ["name", "price", "categoryId"].some(k => !(k in body))) {
    return badRequest(res, "name, price and categoryId are required");
  }
  if ("name" in body && (typeof body.name !== "string" ||
      !body.name.trim() || body.name.length > 150)) {
    return badRequest(res, "Invalid name");
  }
  if ("price" in body && (!Number.isSafeInteger(body.price) || body.price < 0 ||
body.price > 2147483647)) {
    return badRequest(res, "Invalid price");
  }
  if ("categoryId" in body &&
      (!Number.isSafeInteger(body.categoryId) || body.categoryId <= 0 || body.categoryId
> 2147483647)) {
    return badRequest(res, "Invalid categoryId");
  }
  if ("inStock" in body && typeof body.inStock !== "boolean") {
    return badRequest(res, "Invalid inStock");
  }
  // Допускаются только проверенные поля.
  req.body = Object.fromEntries(keys.map(key => [key,
    key === "name" ? body[key].trim() : body[key]
  ]));
  next();
}

export { validId };
export const validateCreate = (req, res, next) => validateBody(req, res, next, false);
export const validateUpdate = (req, res, next) => validateBody(req, res, next, true);
```

Проверяйте наличие категории средствами PostgreSQL FOREIGN KEY: middleware проверяет форму categoryId, но не обязан делать дополнительный запрос к БД. Нарушение ссылки отрабатывает Prisma и централизованный обработчик ошибок.

6. Настройте Product Controller

Файл: controllers/productController.js. Controller преобразует HTTP-запрос в вызов Service и возвращает JSON. Он не обращается к Prisma напрямую.

Файл controllers/productController.js

```
import * as service from "../services/productService.js";

export async function getProducts(req, res) {
  res.json(await service.getAllProducts());
}

export async function getProductById(req, res) {
  const product = await service.findProductById(req.productId);
  if (!product) return res.status(404).json({ message: "Product not found" });
  res.json(product);
}

export async function createProduct(req, res) {
  const product = await service.addProduct(req.body);
  res.status(201).json(product);
}

export async function updateProduct(req, res) {
  const product = await service.updateProductById(req.productId, req.body);
  res.json(product);
}

export async function deleteProduct(req, res) {
  const product = await service.deleteProductById(req.productId);
  res.json(product);
}
```

Передача ошибок

В Express 5 отклонённые Promise из async-обработчиков передаются в обработчик ошибок. Если в старом Controller используются try/catch с next(error), их тоже можно оставить — не смешивайте два способа в одной функции.

7. Подключите маршруты и validation middleware

Файл: routes/productRoutes.js. URL и HTTP-методы остаются прежними; изменяется только подключение проверки данных. В случае уже настроенной validation достаточно убедиться, что она применяется к соответствующим маршрутам.

Файл routes/productRoutes.js

```
import { Router } from "express";
import * as controller from "../controllers/productController.js";
import { validId, validateCreate, validateUpdate } from
"../middleware/productValidation.js";
```

```
const router = Router();
router.get("/", controller.getProducts);
router.get("/:id", validId, controller.getProductById);
router.post("/", validateCreate, controller.createProduct);
router.patch("/:id", validId, validateUpdate, controller.updateProduct);
router.delete("/:id", validId, controller.deleteProduct);
export default router;
```

8. Обработка HTTP-ошибок в приложении

Файл: app.js. Проверьте подключение express.json(), маршрутов и errorHandler именно в этом порядке. Если у вас уже есть общий обработчик, добавьте в него обработку ошибок Prisma, не создавая два конкурирующих обработчика.

Файл app.js — пример для API товаров

```
import express from "express";
import productRoutes from "./routes/productRoutes.js";

const app = express();
app.use(express.json());
app.use("/api/products", productRoutes);

// Неверный JSON (ошибка парсинга) — 400
// P2025 — отсутствующий Product (404)
// P2003 — нарушен FOREIGN KEY категории (400)
app.use((err, req, res, next) => {
  if (err instanceof SyntaxError && err.status === 400 && "body" in err) {
    return res.status(400).json({ message: "Invalid JSON" });
  }
  if (err.code === "P2025") {
    return res.status(404).json({ message: "Product not found" });
  }
  if (err.code === "P2003") {
    return res.status(400).json({ message: "Invalid categoryId or related record" });
  }
  console.error(err);
  res.status(500).json({ message: "Server error" });
});

app.listen(3000, () => console.log("Server started on port 3000"));
```

Не потеряйте предыдущий функционал

Код app.js показан как минимальный самостоятельный вариант. Если в существующем проекте подключены categoryRoutes, глобальная обработка 404, дополнительные middleware или иные маршруты — сохраните их. Обработчик ошибок должен идти после всех маршрутов.

9. Запустите API

Файл package.json: сохраните существующие зависимости и убедитесь, что скрипт запуска использует tsx.

Фрагмент package.json; не заменяйте весь файл

```
{
  "type": "module",
  "scripts": {
    "start": "tsx app.js",
    "dev": "tsx watch app.js",
    "prisma:generate": "prisma generate"
  }
}
```

```
npx prisma validate
npx prisma generate
npm start
```

10. Проверьте все CRUD-операции

Тестируйте через Thunder Client, Postman или аналогичный HTTP-клиент. В примерах используется категория с id = 1; сначала проверьте в PostgreSQL, что она существует.

```
Проверьте доступные категории в БД
SELECT id, name FROM categories ORDER BY id;
```

HTTP	Адрес	Ожидаемый результат
GET	/api/products	200: массив
GET	/api/products/1	200: объект; 404, если отсутствует
POST	/api/products	201: созданный объект
PATCH	/api/products/1	200: изменённый объект
DELETE	/api/products/1	200: удалённый объект

```
POST /api/products — тело JSON
{
  "name": "SSD 2TB",
  "price": 65000,
  "categoryId": 1,
  "inStock": true
}
```

```
PATCH /api/products/:id — тело JSON
{
  "price": 61000,
  "inStock": false
}
```

После POST запишите id созданного товара. Выполните GET по этому id, затем PATCH и DELETE по тому же id. После удаления повторный GET должен вернуть 404. Для DELETE используйте созданный на занятии тестовый товар, а не данные, связанные с заказами.

11. Проверьте ошибки и ограничения

Ситуация	Запрос/условие	Ожидаемый HTTP
Неправильный ID	GET /api/products/abc	400
Товар не существует	GET /api/products/999999999	404
Пустое тело POST	POST /api/products с {}	400
Некорректная цена	POST с price = "много"	400
Нет нужной категории	POST с categoryId несуществующей категории	400 (P2003)
Товар не существует	PATCH /api/products/999999999	404 (P2025)
Товар не существует	DELETE /api/products/999999999	404 (P2025)

Отдельно о целостности данных

Код P2003 соответствует нарушению FOREIGN KEY; конкретный ответ может отличаться в иной схеме БД. Prisma не заменяет валидацию. Если в PostgreSQL есть дополнительные CHECK-ограничения, проверьте их отдельно.

12. Сравните с прежней реализацией через pg

- Найдите старые SELECT, INSERT, UPDATE, DELETE в productService.js.
- Убедитесь, что в Product Service больше нет pool.query() и импортов config/db.js.
- Убедитесь, что в контроллерах нет prisma.product.*.
- Названия endpoints не изменились.
- Проверьте, что данные действительно появляются, изменяются и удаляются в PostgreSQL.

Важно

config/db.js и библиотеку pg пока не удаляйте: PostgreSQL-адаптер @prisma/adapter-pg использует пакет pg, а другие сервисы могут ещё использовать старый способ подключения.

13. Самостоятельная работа

- Добавьте к GET /api/products необязательную фильтрацию по inStock и минимальной цене через Prisma where.
- С помощью select подготовьте компактный список товаров, содержащий только id, name и price.
- Реализуйте CRUD для Category по аналогии с Product без предоставленного готового кода. Продумайте запрет удаления категории, к которой привязаны товары.

- Дополнительное задание: проанализируйте, как заменить операции с таблицами users, orders и order_items на Prisma в следующем модуле.

14. Требования к отчёту

- Цель работы, схема взаимодействия Route → Controller → Service → Prisma → PostgreSQL.
- Код изменённых файлов с указанием путей.
- Скриншоты пяти успешных REST-операций.
- Проверка ошибок 400, 404 и неверного categoryId.
- Скриншот таблицы products до и после изменения записи.
- Краткий вывод: преимущества и ограничения Prisma в сравнении с pg.

15. Контрольные вопросы

- Какой метод Prisma соответствует INSERT?
- В чём разница между findMany() и findUnique()?
- Что означают where, data, orderBy, select и include?
- Почему Model Product не заменяет таблицу PostgreSQL?
- Зачем Product Service нужен include: { category: true }?
- Почему req.params.id необходимо преобразовывать и проверять?
- Почему Prisma Client не заменяет validation middleware?
- Какая ошибка Prisma возникает при update()/delete() отсутствующего товара?
- Почему Route и Controller не должны выполнять запросы Prisma напрямую?
- Как обеспечивается сохранение целостности связи Product → Category?

16. Критерии оценивания

Критерий	Баллы
Сохранены структура проекта и Prisma-подключение	15
Реализован полный CRUD через Product Service	35
Работают корректные HTTP-ответы и валидация	25
Пройдены проверки REST API и ошибок	15
Отчёт и ответы на вопросы	10
Итого	100